Engine plugins

Each game engine is a plugin. Open Factory Studio comes with the M.U.G.E.N plugin (src/engines/mugen). Other engines can be added the same way without touching the core.

Files

Where	What
src/engines/<name>	The plugin's source code
src/engines/<name>/CMakeLists.txt	Builds it with ofs_add_engine_plugin(<name> SOURCES ...)
src/engines/<name>/<name>.json	The plugin's details
data/engines/<name>	The engine's data, installed next to the program's data

When the program runs, it loads the plugin from <appdir>/engines/<name>/ofs_<name>.dll (.so on Linux, .dylib on macOS) and its data from <datadir>/engines/<name>. Users can add or replace data files in <userdir>/engines/<name>.

The json file

```
{
  "name": "mugen",
  "displayName": "M.U.G.E.N",
  "apiVersion": 1,
  "extensions": ["def", "sff", "air", "cns", "st", "cmd", "snd", "fnt", "act"]
}
```

apiVersion must be the same as OFS::EngineManager::ApiVersion. Plugins made for another version are skipped and reported. The version goes up whenever the OFS::Engine interface changes in a way old plugins can't handle.

The code

A plugin implements OFS::EnginePlugin and returns a subclass of OFS::Engine (src/core/engine/Engine.h):

```
class MyPlugin : public QObject, public OFS::EnginePlugin
{
    Q_OBJECT
    Q_PLUGIN_METADATA(IID OFS_ENGINE_PLUGIN_IID FILE "myengine.json")
    Q_INTERFACES(OFS::EnginePlugin)
public:
    OFS::Engine* createEngine(QObject* parent) override { return new MyEngine(parent); }
};
```

Plugins link to ofscore and have to be built with the same compiler and Qt version as the program.

The engine's data folder

An engine's data folder can have any of these:

Path	What it is
config.xml	The engine's default settings. The user's copy is <userdir>/<name>_config.xml (AppContext::engineConfig())
syntax.xml	The syntax database for the code editors
completer.xml	Word lists for code completion
sctrls	State controller templates
snippets	Code snippets
palFX	PalFX templates
presets	Lua presets that build the engine's toolbars and panels
iconsets	Icons for the engine's own actions
layouts	Layouts for HTML export
debug.xml	Where Debug > Start on Engine finds the game's values in the engine's memory, for each engine version (the M.U.G.E.N file explains the format)
scripts	Lua scripts: project templates, tools and generators
docs	The reference pages shown in the editor

The M.U.G.E.N syntax database, completer, snippets, state controller templates, PalFX templates and reference pages are explained in the Syntax database document.

Options page

An engine can add its own tab to Tools > Options. It registers a page factory under its name (src/core/options/OptionsPage.h):

```
OFS::OptionsPageRegistry::registerPage("myengine", [](OFS::Config& engineConfig, QWidget* parent) {
  return new MyEngineOptionsPage(engineConfig, parent);
});
```

The page gets the engine's settings and shows up when the engine is picked at the bottom of the dialog. The M.U.G.E.N page is built into the program itself.

Archive plugins

Plugins in extensions/<name> next to the program add archive formats to Project > Open from archive and to the Archive class in scripts. Such a plugin implements OFS::ExtensionPlugin (src/core/project/Archives.h) and returns OFS::ArchiveFormat objects: the format's name and file extensions, and how to list, read and extract what's inside. The RAR plugin (src/extensions/archive_rar, built with OFS_WITH_RAR) is one of them. To build your own, use ofs_add_extension_plugin() from cmake/OfsHelpers.cmake.